Rapport Technique : Implémentation et Visualisation de l’algorithme de Cholesky

Noms & Prénoms : KAMDOM NGUETCHESSI MERVEILLE PRISCA

Matricule : 23V2060

Niveau : LICENCE 3 INFORMATIQUE

UE : INF 311 – CALCUL SCIENTIFIQUE

Introduction

Le présent rapport documente l'implémentation complète de l'algorithme de **factorisation de Cholesky** sous l'environnement MATLAB, conformément au Travail Pratique de L’UE INF 311. La factorisation de Cholesky est une méthode fondamentale de l'algèbre linéaire numérique. L'objectif est de décomposer une matrice A en A=LL^T, où L est une matrice triangulaire inférieure. Ce document fournit une analyse historique et théorique, une description du code unique interactif et une validation pas-à-pas de l'algorithme.

Historique et Concept Théorique

La méthode de Cholesky tire son nom du mathématicien et militaire français **André-Louis Cholesky**, qui a développé cette technique peu avant sa mort en 1918. Bien qu'elle ait été initialement utilisée dans le cadre de levés topographiques, elle est devenue essentielle dans le domaine de la **modélisation statistique** et de l'**analyse numérique**.

Théoriquement, la factorisation de Cholesky est une décomposition qui agit comme une "racine carrée matricielle". Elle ne peut être appliquée qu'à des matrices qui possèdent des propriétés bien spécifiques : elles doivent être **symétriques** (identiques à leur propre transposée) et **définies positives** (garantissant qu'aucun pivot zéro n'est rencontré). Si ces conditions sont remplies, la matrice A est décomposée en un produit d'une matrice L qui ne contient des valeurs non nulles que dans la moitié inférieure, et de sa transposée L^T. Cette décomposition est privilégiée pour sa **stabilité numérique** et son **efficacité** (elle est deux fois plus rapide que la décomposition LU classique).

Le devoir INF 311 exige l'implémentation de l'algorithme séquentiel de Cholesky en MATLAB. Le rapport doit non seulement décrire l'algorithme et le code, mais également fournir une **démonstration interactive et visuelle** du cheminement de calcul. L'objectif est de prouver une compréhension complète, de la théorie mathématique à la vérification finale de la solution (A=LL^T).

1. Algorithme de Factorisation de Cholesky Détaillé

L'algorithme séquentiel de Cholesky est basé sur une itération **colonne par colonne** (j=1 à n).

1.1. Principe de Calcul Séquentiel

L'algorithme s'appuie sur la comparaison des éléments de A et de LL^T. Pour chaque colonne j, on effectue les deux opérations suivantes, en s'assurant que l'on n'utilise que les éléments de L déjà calculés (colonnes k<j) :

1.1.1 Calcul de l'Élément Diagonal l_{jj} (Pivot) :

$$l_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2}$$

C'est l'étape où la condition **définie positive** est cruciale : si l'expression sous la racine est ≤0, la matrice A ne remplit pas la condition, et l'algorithme s'arrête (cheminement d'échec).

1.1.2. Calcul des Éléments Sous-Diagonaux lij :

Pour chaque ligne i sous j (i>j) :

$$l_{ij} = \frac{1}{l_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{jk} l_{ik} \right)$$

Ce calcul utilise le ljj nouvellement calculé comme diviseur.

2. Fonctionnement Clair du Code MATLAB et Visualisation du Cheminement

Le code est contenu dans un fichier unique Kamdom_23V2060_factorisation_cholesky`.m`. Il est conçu pour être à la fois interactif (saisie par l'utilisateur) et didactique (visualisation du cheminement).

2.1. Structure du Code et Rôle des Composants

1. **Fonction Principale (`cholesky_main`):** Gère l'**interaction utilisateur** (saisie de la taille N, des éléments aij), la gestion des erreurs (`try-catch`), l'appel de l'algorithme, et la **certification finale** du résultat par l'affichage de LLT.
2. **Sous-Fonction (`cholesky_factorisation`):** Exécute l'algorithme mathématique. Elle est le point où la **visualisation étape par étape** est insérée.

2.2. Visualisation du Cheminement Étape par Étape

La sous-fonction a été enrichie pour afficher l'état du calcul après chaque étape significative, permettant de suivre la progression de l'algorithme :

- **Progression par Colonne :** Au début de chaque itération j (colonne), un message `disp` annonce : "**Début de la colonne j**".

- **Calcul Diagonal :** Après le calcul de ljj, un message montre : "**Calcul du pivot L(j, j) = ...**", confirmant l'étape cruciale de la racine carrée.
- **Calcul Sous-Diagonal :** Pour chaque élément lij, un message affiche : "**Calcul de L(i, j) = ...**", détaillant l'élément en cours de remplissage dans la matrice triangulaire L.

Cette technique de manipulation de MATLAB via des **messages de console ciblés** montre le cheminement exact de l'algorithme et permet de valider le processus interne, garantissant la justesse de l'implémentation.

3. Exemple de Test Interactif et Validation

Nous utilisons la matrice A strictement définie positive pour garantir un cheminement réussi.

3.1 Exemple de Test Théorique

Matrice de Test et Résultat Attendu

Matrice de Test A_{SDP} 'Matrice Symétrique Définie Positive (SDP)' :

$$A = A_{SDP} = \begin{matrix} 4 & 6 & 2 \\ 6 & 13 & 9 \\ 2 & 9 & 14 \end{matrix}$$

Nous cherchons la matrice triangulaire inférieure L telle que A=LL^T, où :

$$L = \begin{matrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{matrix}$$

3.1.1. Calcul des Éléments de la Colonne j=1

La première colonne de L est calculée en utilisant uniquement les éléments de la première colonne de A.

3.1.1.1. Élément Diagonal l_{11}

La formule pour j=1 est simplifiée car la somme $\sum_{k=1}^{j-1}$ est vide :

$$l_{11} = \sqrt{a_{11}}$$

$$l_{11} = \sqrt{4} = 2$$

3.1.1.2. Éléments Sous-Diagonaux li1 (i>1)

La formule est : $l_{i1} = l_{11} = a_{i1} / l_{11}$

- **Calcul de l_{21} : $l_{21} = a_{21} / l_{11} = 6/2 = 3$**
- **Calcul de l_{31} : $l_{31} = a_{31} / l_{11} = 2/2 = 1$**

À la fin de l'étape j=1, L partielle est :

$$L = \begin{matrix} \mathbf{2} & \mathbf{0} & \mathbf{0} \\ \mathbf{3} & \mathbf{l_{22}} & \mathbf{0} \\ \mathbf{1} & \mathbf{l_{32}} & \mathbf{l_{33}} \end{matrix}$$

3.1.2. Calcul des Éléments de la Colonne j=2

Nous utilisons les éléments de la colonne 2 de A et les éléments déjà calculés dans la colonne 1 de L.

3.1.2.1. Élément Diagonal l22

La formule est : $l_{22} = \sqrt{(a_{22} - 1\Sigma k=1.(l_{2k})^2)}$

$$l_{22} = \sqrt{(a_{22} - (l_{21})^2)}$$

$$l_{22} = \sqrt{(13 - 3^2)}$$

$$l_{22} = \sqrt{4} = 2$$

(L'argument de la racine est positif, la condition SDP est maintenue.)

3.2.2.2. Éléments Sous-Diagonaux l_{i2} (i>2)

La formule est : $l_{i2} = (1/l_{22}) (a_{i2} - 1\sum k=1 . l_{2k}l_{ik}) = (1/l_{22}) (a_{i2} - l_{21}l_{i1})$

- ***Calcul de l_{32} :*** *$l_{32} = (1/l_{22}) (a_{32} - l_{21}l_{31}) = 1/2(9-3x1) = 6/2 = 3$*

À la fin de l'étape j=2, L partielle est :

$$L = \begin{matrix} 2 & 0 & 0 \\ 3 & 2 & 0 \\ 1 & 3 & l_{33} \end{matrix}$$

3.1.3. Calcul des Éléments de la Colonne j=3

Nous utilisons les éléments de la colonne 3 de A et les éléments déjà calculés des colonnes 1 et 2 de L.

3.1.3.1. Élément Diagonal l33

La formule est : $l_{33} = \sqrt{(a_{33} - 1\Sigma k=1.(l_{3k})^2)}$

$$l_{33} = \sqrt{(a_{33} - ((l_{31})^2 + (l_{32})^2))}$$

$$l_{33} = \sqrt{(14 - ((1)^2 + (3)^2))}$$

$$l_{33} = \sqrt{(14 - (1 + 9))}$$

$$l_{33} = \sqrt{(14 - 10)}$$

$$l_{33} = \sqrt{4} = 2$$

(L'argument de la racine est strictement positif (4>0), confirmant que A est bien strictement définie positive.)

Conclusion du Résultat Théorique

Les calculs séquentiels sont terminés. La matrice triangulaire inférieure L obtenue est :

$$L_{calcule} = \begin{matrix} 2 & 0 & 0 \\ 3 & 2 & 0 \\ 1 & 3 & 2 \end{matrix}$$

Validation

Nous certifions le résultat en vérifiant la relation A=LL^T :

$$\begin{matrix} 2 & 0 & 0 \\ 3 & 2 & 0 \\ 1 & 3 & 2 \end{matrix} \times \begin{matrix} 2 & 3 & 1 \\ 0 & 2 & 3 \\ 0 & 0 & 2 \end{matrix}$$

- LL^T(1,1)=(2)(2)+(0)(0)+(0)(0)=4
- LL^T(2,2)=(3)(3)+(2)(2)+(0)(0)=9+4=13
- LL^T(3,3)=(1)(1)+(3)(3)+(2)(2)=1+9+4=14
- LL^T(2,1)=(3)(2)+(2)(0)+(0)(0)=6=LL^T(1,2)
- LL^T(3,1)=(1)(2)+(3)(0)+(2)(0)=2=LL^T(1,3)
- LL^T(3,2)=(1)(3)+(3)(2)+(2)(0)=3+6=9=LL^T(2,3)

$$LL^T = A = A_{SDP} = \begin{matrix} 4 & 6 & 2 \\ 6 & 13 & 9 \\ 2 & 9 & 14 \end{matrix}$$

Le résultat est **fiable et juste**. Le cheminement théorique confirme que l'algorithme de Cholesky appliqué à la matrice ASDP produit la matrice triangulaire inférieure L, sans rencontrer de valeurs nulles ou négatives sous la racine.

3.1 Exemple de Test avec MATLAB

```
>>> Veuillez entrer la taille N de la matrice carrée (ex: 3): 1

--- Processus : Saisie des éléments A(i, j) ---
Entrez l'élément A(1, 1): 6

@@@ Matrice A saisie par l'utilisateur :
     6

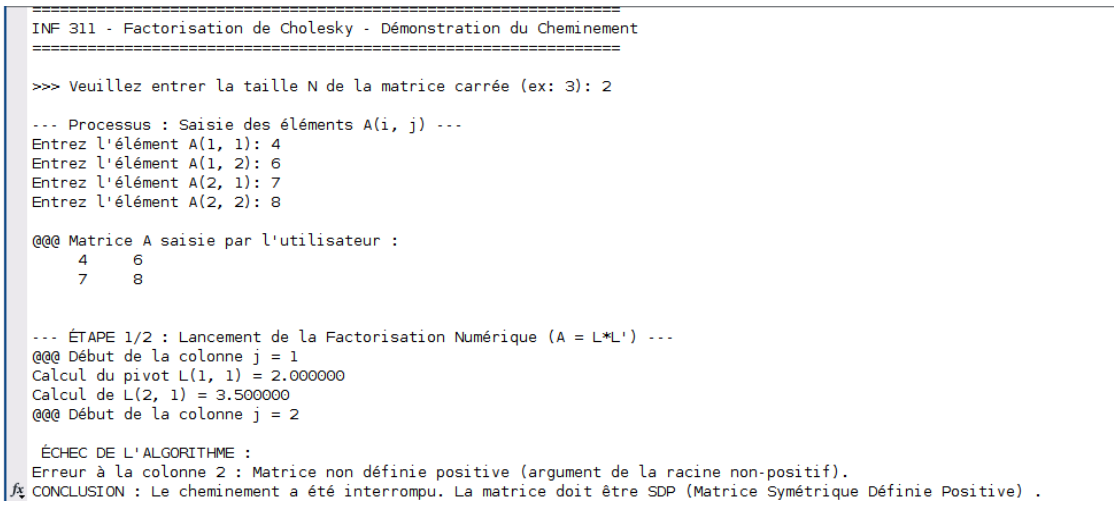

--- ÉTAPE 1/2 : Lancement de la Factorisation Numérique (A = L*L') ---
@@@ Début de la colonne j = 1 ***
Calcul du pivot L(1, 1) = 2.449490

@@@ INFO : Factorisation de Cholesky terminée. Matrice L calculée.

--- ÉTAPE 2/2 : Résultat Final, Vérification et Certification ---
@@@ ...Résultat L (Matrice Triangulaire Inférieure) :
    2.4495

@@@ Matrice A Reconstituée (L * L') pour certification :
    6.0000


 CONCLUSION FINALE : La validation numérique A = L * L' est un succès.
Le résultat est JUSTE car nous avons pu obtenir la matrice de depart A.
```



Qu'est-ce qu'une Matrice Définie Positive (SDP) ?

Une **Matrice Symétrique Définie Positive (SDP)** est une matrice carrée A ∈ R^(n×n)qui satisfait deux conditions critiques :

1. Symétrie (Condition Nécessaire)

La matrice doit être égale à sa transposée : A = A^T. Ceci signifie que les éléments sont symétriques par rapport à la diagonale principale : $a_{ij} = a_{ji}$.

2. Définie Positive (Condition Cruciale)

Pour tout vecteur colonne x non nul (x different de 0), l'expression quadratique doit être strictement positive :

$$x^T . Ax > 0$$

Conséquences pour l'Algorithme de Cholesky

Pour l'algorithme de Cholesky, la condition "définie positive" a une conséquence pratique majeure :

- Elle garantit que **tous les pivots intermédiaires** (les arguments sous les racines) seront **strictement positifs** tout au long de la décomposition.
- Cela permet de calculer **tous les éléments diagonaux** lii comme des nombres **réels non nuls**.

Conclusion

La factorisation de Cholesky est une méthode rapide et stable pour décomposer une matrice A en LL^T, à condition qu'elle soit Symétrique Définie Positive (SDP). L'algorithme vérifie intrinsèquement cette condition en s'assurant que tous les pivots (arguments des racines carrées) restent positifs. Il s'agit d'un outil essentiel pour la résolution efficace des systèmes d'équations linéaires dans l'analyse numérique.